

Chapter 14. Pod Scheduling

The *scheduler* is the cluster component responsible for deciding which node to select for running a Pod. In this chapter, you will learn about the general scheduling algorithm and the Kubernetes concepts that let you express soft and hard requirements for influencing the scheduling decisions.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Configure Pod admission and scheduling (limits, node affinity, etc.)
-

Pod Scheduling Algorithm

Initially a Pod created by an end user does not have a node assigned. That's the job of the scheduler cluster component. The scheduler runs in a scheduling loop, watching for unscheduled Pods. It will then evaluate available nodes to choose the most suitable one for the Pod.

Finding a fitting node follows a two-step approach: *filtering* and *scoring*. The filtering step determines the list of nodes feasible for running a Pod (e.g., by checking the available hardware capacity). The scoring step ranks the remaining nodes to select the most suitable to run the workload. Scheduling decisions include resource requirements, affinity and anti-affinity specifications, and many more. You can find a visualization of the Pod scheduling algorithm in [Figure 14-1](#).

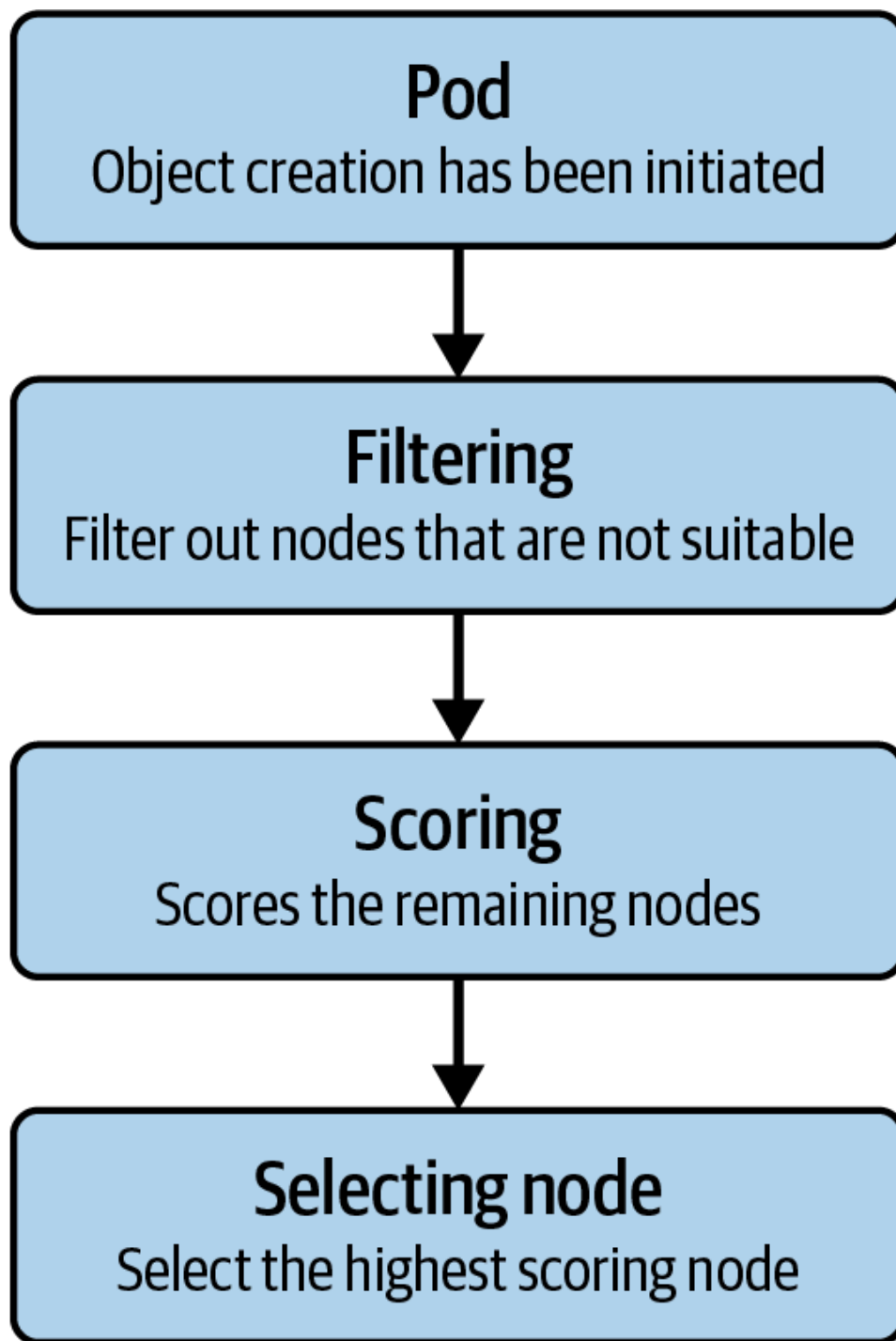


Figure 14-1. Pod scheduling algorithm

Pods and their container(s) can define requirements that help with determining the node. If the requirements cannot be met by any of the nodes, then the Pod stays unscheduled until the scheduler checks again. Otherwise, the scheduler picks the node with the highest score and assigns the Pod to it.

Setting Up a Multi-Node Development Cluster

The effects of scheduling requirements are best explained by demonstrating them on a multi-node cluster. For the remainder of this chapter, I am

going to use a cluster with one control plane node and three worker nodes, as shown here:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
multi-node	Ready	control-plane	2m33s	v1.32.2
multi-node-m02	Ready	<none>	2m22s	v1.32.2
multi-node-m03	Ready	<none>	2m15s	v1.32.2
multi-node-m04	Ready	<none>	2m9s	v1.32.2

Setting up a multi-node cluster with a Kubernetes environment is pretty easy to achieve. Most development cluster implementations like [minikube](#) and [kind](#) offer an option to initiate more than a single node. The following command creates a four-node cluster with the name prefix `multi-node` using minikube:

```
$ minikube start --kubernetes-version=v1.32.2 --nodes=4 -p multi-node
```

Refer to the documentation of the Kubernetes development cluster of your choice for more information on applicable configuration options.

Determining the Node a Pod Runs On

The prerequisite for being able to schedule a Pod is that the scheduler is functional. The scheduler process runs in a Pod on the control plane node. You can use the following command to find the scheduler Pod. Make sure that the Pod has the `Running` status:

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
kube-scheduler-multi-node	1/1	Running	0	5m10s

You can find out which node a Pod runs with the `kubectl get` or `kubectl describe` command. The following commands show variations of their usage for a Pod named `nginx`:

```
$ kubectl get pod nginx -o=wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx	1/1	Running	0	3m49s	10.244.2.2	multi-node-m03

```
$ kubectl get pod nginx -o yaml | grep nodeName:
```

```
  nodeName: multi-node-m03
```

```
$ kubectl describe pod nginx
```

```
Name:                nginx
```

```
Namespace:      default
Priority:        0
Service Account: default
Node:           multi-node-m03/192.168.49.4
...
```

Pod Scheduling Options

The scheduler does a reasonably good job of assigning a Pod to a feasible node. Under certain conditions, you may want to restrict which node a Pod can run on, or define a preferred selection criteria. This is usually expressed by using label selection.

Kubernetes offers a variety of Pod scheduling options, each of which can be combined with one another. In this chapter, I am going to discuss the following concepts, however, there are more you can select from:

Node selector

A hard requirement to determine which node the Pod needs to run on

Node affinity and anti-affinity

A more flexible requirement for defining hard or soft requirements for node selection

Taints and tolerations

A way to safeguard specific nodes from scheduling Pods on them based on conditions and requirements

Pod topology spread constraints

Defines how to spread Pods across different topologies, i.e., regions and zones

Working with Node Selectors

The node selector defines a hard requirement for scheduling a Pod on specific nodes. To use the node selector, label one or many nodes with a specific label key-value pair. When defining a Pod in a YAML manifest, select the label from the Pod via the attribute `spec.nodeSelector`.

A typical example for using the node selector is to ensure that Pods end up running on a node with specific hardware. Input/output-intensive ap-

plications that require fast disk access, e.g., as supported by solid-state drive (SSD) volumes, could make good use of this concept.

Figure 14-2 illustrates a cluster with three nodes. Node 1 has the label `disk=ssd` and Node 2 has the label `disk=hdd`. Pod 1 can only be scheduled on Node 1, as it defines the matching node selector, and therefore on none of the other nodes. Pod 2 cannot be scheduled on any of the nodes, as the node selector doesn't match any of the nodes' labels. Pod 1 and Pod 2 cannot be scheduled on Node 3.

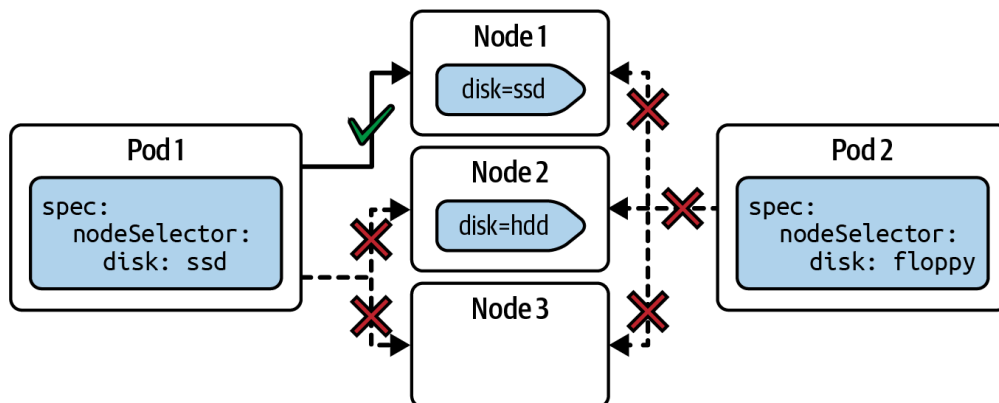


Figure 14-2. Node selector scenarios

Labeling a Node

Following the described use case of running applications only on nodes that provide SSD volume access, we'll first need to assign a specific label key-value pair. The following command uses the imperative `kubectl label` command to assign the label `disk=ssd` to the node named `multi-node-m03`:

```
$ kubectl label node multi-node-m03 disk=ssd
node/multi-node-m03 labeled
```

You can find the assigned labels for all nodes when listing them with the `--show-labels` option:

```
$ kubectl get nodes --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
multi-node	Ready	control-plane	14m	v1.32.2	...
multi-node-m02	Ready	<none>	14m	v1.32.2	...
multi-node-m03	Ready	<none>	14m	v1.32.2	...
multi-node-m04	Ready	<none>	14m	v1.32.2	...,disk=ssd,...

Next up, you will need to select this label from the Pod that should be scheduled on the node `multi-node-m03`.

Assigning a Node Selector to a Pod

The only addition to the typical structure is the definition of the `spec.nodeSelector` attribute, as shown in [Example 14-1](#).

Example 14-1. Assigning a node selector

```
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  nodeSelector:           ❶
    disk: ssd             ❷
  containers:
  - name: nginx
    image: nginx:1.27.1
```

❶ Defines a node selector for the Pod

❷ The label key-value pair used to determine a suitable node

NODE SELECTOR LIMITATIONS

The node selector is not limited to a single label key-value pair. Selecting a map of labels is completely valid.

The Pod with this definition can only be scheduled on the node that provides the matching label. You can verify that the Pod runs on the expected node with the same command described in [“Determining the Node a Pod Runs On”](#).

Working with Node Affinity and Anti-Affinity

In Kubernetes, the `spec.nodeSelector` field is used to define strict scheduling constraints—it allows you to specify hard requirements that a node must satisfy for a Pod to be scheduled on it. While simple and straightforward, the node selector is limited to exact key-value label matches and does not support advanced logic.

For more flexible and expressive scheduling rules, you should use node affinity, defined under `spec.affinity.nodeAffinity` in the Pod speci-

fication. Node affinity allows you to match nodes using label selector expressions, enabling more complex criteria such as [logical operators](#) (`In` , `NotIn` , `Exists` , etc.) and prioritized preferences.

Coming back to the previous use case, you may want to run Pods on nodes that support SSD-based volumes *or* volumes with slower storage devices if the primary preference cannot be fulfilled.

[Figure 14-3](#) shows the node affinity concept in action. In this scenario, Pod 1 can be scheduled on Node 1 or Node 2 based on the defined set-based label selection.

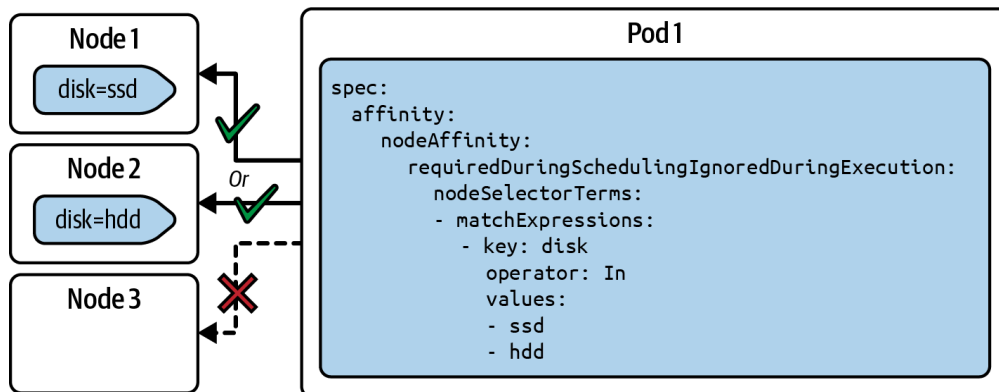


Figure 14-3. Node affinity scenarios

Node anti-affinity is useful in situations where you want to prevent Pods from being scheduled on specific nodes. This is particularly helpful in high-availability situations where you want to spread Pods across different zones or regions.

Assigning a Node Affinity to a Pod

In short, node affinity effectively replaces the node selector for most use cases, offering greater precision and control over workload placement.

[Example 14-2](#) shows the same Pod definition we saw earlier, but in this case it allows for placing the Pod on a node where the assigned label key-value pair is `disk=ssd` or `disk=hdd`.

Example 14-2. Assigning node affinity

```
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
```

```

- matchExpressions:
  - key: disk
    operator: In
    values:
      - ssd
      - hdd
containers:
- name: nginx
  image: nginx:1.27.1

```

③
④
④
④
④
④

- ❶ Defines a node affinity for the Pod
- ❷ The node affinity type that will be adhered to when scheduling the Pod
- ❸ The expression for finding matching nodes
- ❹ A set-based label requirement

Node Affinity Types

Beyond supporting set-based expressions, node affinity also introduces specific types that control when the rules are applied. One commonly used type is `requiredDuringSchedulingIgnoredDuringExecution`. This setting means that the affinity rules are strictly enforced only at the time of scheduling—when the Pod is initially assigned to a node. Once the Pod is running, any changes to the node affinity rules are ignored and will not trigger rescheduling.

This isn't the only available node affinity type. [Table 14-1](#) shows the list. Types starting with `requiredDuringScheduling` express a hard requirement, and types starting with `preferredDuringScheduling` express a soft requirement, a preference which the schedule doesn't have to adhere to in case no fitting node can be determined.

Table 14-1. Node affinity types

Type	Description
<code>requiredDuringSchedulingIgnoredDuringExecution</code>	Rules that must be met for a Pod to be scheduled onto a node
<code>preferredDuringSchedulingIgnoredDuringExecution</code>	Rules that specify preferences that the scheduler will try to enforce but will not guarantee

At the time of writing, none of the node affinity types support modifying an already scheduled Pod, as indicated by the `IgnoredDuringExecution` suffix. The Kubernetes team may decide to change that in a future release.

Node Affinity Operators

In the previous example, you saw one of the node affinity [operators](#) in action: the `In` operator. The `In` operator is an operator that defines a requirement to find a label value as part of a given set of strings. You can select other operators to define node affinity requirements, as listed in [Table 14-2](#).

Table 14-2. Node affinity operators

Operator	Behavior
<code>In</code>	A node has an assigned label value in the given set of strings.
<code>NotIn</code>	Only those nodes are selected that do not have an assigned label value in the given set of strings.
<code>Exists</code>	A node has a label key assigned to it that matches the given string.
<code>DoesNotExist</code>	A node does not have a label key assigned to it that matches the given string.
<code>Gt</code>	Selects nodes where the label's value is numerically greater than the specified value, like selecting nodes with more than 8 CPUs using “cpu-count Gt 8.”
<code>Lt</code>	Selects nodes where the label's value is numerically less than the specified value, like selecting nodes with less than 16 GB memory using “memory-gb Lt 16.”

There are two operators, `NotIn` and `DoesNotExist`, that negate the selective effects of their counterparts `In` and `Exists`. Those operators are used to define a node anti-affinity behavior.

Assigning a Node Anti-Affinity to a Pod

Node anti-affinity rules are typically used to prevent certain Pods from being scheduled on the same nodes, based on labels. An essential tool for defining anti-affinity behavior is the operator. [Example 14-3](#) uses the `NotIn` operator to repel Pods from a set of nodes with the given label values.

Example 14-3. Assigning node anti-affinity

```
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disk
                operator: NotIn
                values:
                  - ssd
                  - ebs
  containers:
    - name: nginx
      image: nginx:1.27.1
```

❶

❶ Defines a node anti-affinity by way of using a negating condition

In essence, node anti-affinity does not require you to learn a new API or new attributes when defining a Pod. Its usage primary boils down to the operator you select to define the node affinity rule.

Working with Taints and Tolerations

Similar to node anti-affinity, taints and tolerations represent another way in Kubernetes to influence where Pods can be scheduled, but they serve different purposes and work in fundamentally different ways. Node anti-affinity is meant to be used to spread or separate workload across nodes, whereas taints and tolerations are used for node isolation and workload protection.

The main purpose of taints and tolerations is to prevent Pods from being scheduled on a node unless they explicitly *tolerate* that node's taint. You'd add the taint to a node to say, "don't schedule anything here unless it tolerates the taint." Then in the Pod, you'd add a toleration if you want it to become schedulable on the tainted node.

A typical use case for using taints and tolerations is the need to ensure that Pods are not scheduled on control plane nodes. Control plane nodes use the taint `node-role.kubernetes.io/control-`

`plane:NoSchedule` to prevent Pods from being scheduled on them unless they provide a corresponding toleration.

In [Figure 14-4](#), you can see a scenario that demonstrates the use of taints and tolerations. Node 1 and Node 3 tolerate Pod 1 and Pod 2. Node 2 accepts Pod 1, as it provides the matching toleration, but can also be scheduled on Node 1 and Node 3, as they do not define a taint.

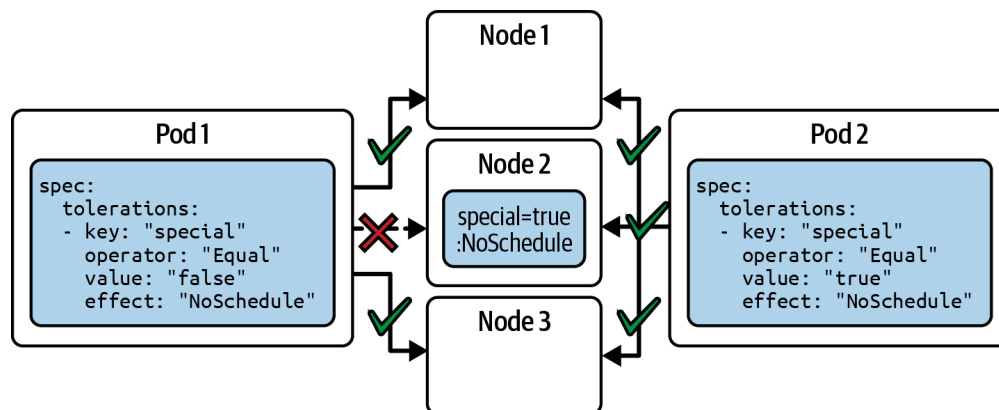


Figure 14-4. Taints and tolerations scenarios

Let's demonstrate the process of adding a taint to a node and a toleration to a Pod by example.

Tainting a Node

A taint on a node marks it as unsuitable for certain Pods unless those Pods explicitly state they can tolerate it. A taint consists of three parts—key, value, and effect—formatted as `key=value:effect`. The key and value portions represent a simple, free-form key-value pair, similar to a label assignment. The effect describes the runtime treatment of the taint by the scheduler.

Use the imperative `kubectl taint` command to add a taint to a node. The following command adds the taint `special=true:NoSchedule` to the node named `multi-node-m02`:

```
$ kubectl taint node multi-node-m02 special=true:NoSchedule  
node/multi-node-m02 tainted
```

The scheduler now considers this taint during Pod placement. To view the assigned taints of a node, run the `kubectl get` or `kubectl describe` command. The following command renders the YAML representation of the node `multi-node-m02` and then finds the relevant information in the output by combining it with the Linux `grep` command:

```
$ kubectl get node multi-node-m02 -o yaml | grep -C 3 taints:
...
spec:
  taints:
  - effect: NoSchedule
    key: special
    value: "true"
```

Taint Effects

The taint shown in the previous example used the `NoSchedule` effect, which is a hard block for any Pod that doesn't come with the relevant toleration. You can choose from other taint effects, explained in [Table 14-3](#).

Table 14-3. Taint effects

Effect	Description
<code>NoSchedule</code>	Unless a Pod has matching toleration, it won't be scheduled on the node.
<code>PreferNoSchedule</code>	Try not to place a Pod that does not tolerate the taint on the node, but it is not required.
<code>NoExecute</code>	Evict Pod from node if already running on it. No future scheduling on the node.

In short, you can think of the available taint effects and their runtime enforcement in the following way. The effect `NoSchedule` is a hard block. `PreferNoSchedule` offers a soft suggestion. Lastly, `NoExecute` not only blocks Pods without the corresponding toleration but also evicts already running Pods that can't fulfill the requirement.

Assigning a Toleration to a Pod

To enable a Pod to run on a tainted node, you must add a toleration to the Pod specification that precisely matches the key, value, and effect of the node's taint. This tells the scheduler that the Pod is allowed to tolerate the taint and may be placed on the node despite the restriction.

[Example 14-4](#) shows a matching toleration assigned to a Pod for the taint `special=true:NoSchedule`.

Example 14-4. Assigning a toleration

```
apiVersion: v1
kind: Pod
```

```

metadata:
  name: app
spec:
  tolerations:
    - key: "special"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"
  containers:
    - name: nginx
      image: nginx:1.27.1

```

- ❶ The attribute that lets you define one or more tolerations.
- ❷ The key of the toleration that needs to match the taint.
- ❸ A toleration “matches” a taint if the keys are the same and the effects are the same.
- ❹ The value of the toleration that needs to match the taint.
- ❺ The runtime effect.

The usage of the effect in a toleration is required in most practical cases. If your toleration is missing the effect, it won’t match any taint. Leaving off the effect is considered optional only when the operator `Exists` is used and you want to tolerate all taints with a specific key (regardless of value). Nevertheless, even in this scenario, specifying the effect is recommended.

Working with Pod Topology Spread Constraints

Pod topology spread constraints control how Pods are distributed across your cluster to improve availability, resilience, and resource utilization. They define rules for how Pods of a certain group (typically in the same Deployment) should be spread across topology domains (like zones, nodes, or racks). You can define the Pod topology spread constraint in the Pod API with the attribute `spec.topologySpreadConstraints`.

Assigning a Topology Spread Constraint to a Pod

[Example 14-5](#) shows an example for a Deployment YAML definition with six replicas of an app and ensures that they are evenly spread across

availability zones.

Example 14-5. Assigning a topology spread constraint to a Pod

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 6
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: web
      containers:
        - name: nginx
          image: nginx:1.27.1
```

- ❶ The difference in Pod count between zones must not exceed one.
- ❷ Uses the node label, in this case the reserved label for defining the Pods, spread across availability zones.
- ❸ What to do if a spread cannot be satisfied.
- ❹ Applies the spread rule only for Pods with the given label(s).

Topology Spread Constraint Effects at Runtime

There are a couple of important facts to mention on how the concept behaves at runtime. Pod topology spread constraints only affect newly-scheduled Pods; they do not rebalance existing Pods. You can define multiple constraints, e.g., a spread by zones and node hostnames. Be aware that the use of the concept can lead to unschedulable Pods if constraints are too strict and resources are limited.

Summary

Kubernetes Pod scheduling is the process of assigning Pods to available nodes in a cluster based on various criteria. By default, the Kubernetes scheduler considers resource requirements, node availability, and constraints like taints, tolerations, and node selectors.

Developers can influence scheduling using node affinity, which expresses preferences for particular node labels, and Pod affinity/anti-affinity, which controls whether Pods are colocated or separated. Taints and tolerations allow nodes to repel certain Pods unless those Pods explicitly tolerate the taint. Topology spread constraints help evenly distribute Pods across failure domains like zones or nodes.

Exam Essentials

Be able to identify the node a Pod runs on

As an end user to Kubernetes, you can easily find out which node a Pod runs on. Become familiar with the relevant `kubectl` commands that give you access to this information. During the exam, you may be asked which Pods run on which nodes of the cluster.

Understand the ins and outs of Pod scheduling options

You'll need to be familiar with a wide range of Pod scheduling concepts. Tasks in the exam may ask you select the most suitable concept to define soft or hard requirements for specific scheduling scenarios. Most likely, the Pod scheduling concept is spelled out explicitly, and you will need to be able to apply the syntax appropriately.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Inspect the existing nodes and their assigned labels. Pick one available node and label it with the key-value pair `color=green`. Pick a second node and label it with the key-value pair `color=red`.

Define a Pod with the image `nginx:1.27.1` in the YAML manifest file `pod.yaml`. Use the `nodeSelector` assignment to schedule the Pod on the node with the label `color=green`. Create the Pod and ensure that the correct node has been used to run the Pod.

Change the Pod definition to schedule it on nodes with the label `color=green` or `color=red`. Verify that the Pod runs on the correct node.

2. Define a Pod with the image `nginx:1.27.1` in the YAML manifest file *pod.yaml*. Create the Pod and check which node the Pod is running on. Add a taint to the node. Set it to `exclusive=yes`. The effect should be `NoExecute`.

Modify the live Pod object by adding the following toleration: it should be equal to the taint key-value pair and have the effect `NoExecute`.

Observe the running behavior of the Pod. If your cluster has more than a single node, where do you expect the Pod to run?

Remove the taint from the node. Do you expect the Pod to still run on the node?